

So sánh hiệu năng công cụ thu thập dữ liệu mã nguồn mở trên website Việt Nam

Sep 25, 2026 · @Le Nhat Tung

1. Mục tiêu và sản phẩm

Sau bài này, mỗi nhóm tự viết được cùng một chương trình thu thập bằng 4 công cụ (Requests + BeautifulSoup, Scrapy, Selenium, Playwright), đo được thời gian và RAM của từng công cụ, và có đủ số liệu để viết một bài báo gửi hội thảo trong nước. Dữ liệu thực hành lấy từ website Nhà thuốc Long Châu.

Câu hỏi nghiên cứu của đề tài

- RQ1: Mỗi công cụ mất bao nhiêu thời gian và bao nhiêu RAM để thu thập cùng 1.000 trang?
- RQ2: Mỗi công cụ lấy đúng dữ liệu bao nhiêu phần trăm trên trang tĩnh, trang render bằng JavaScript và trang cuộn vô hạn / nút "Xem thêm"?
- RQ3: Viết cùng một chương trình bằng mỗi công cụ cần bao nhiêu dòng code? Công cụ nào phù hợp với người mới học?

Yêu cầu đầu vào: biết chạy Python trong Jupyter/VS Code, biết vòng lặp `for`, hàm, và các lệnh SQLite đã học ở chương 03. Không cần kiến thức phân tích dữ liệu.

Thời lượng gợi ý: 4 buổi thực hành trên lớp + 4–6 tuần làm thí nghiệm và viết bài.

Buổi	Nội dung	Sản phẩm nộp
1	Lý thuyết, cài đặt, khảo sát website, Bước 1–2	Danh sách 1.000 URL sản phẩm trong SQLite
2	Requests + BeautifulSoup, Scrapy (Bước 3–4)	2 script chạy được trên 20 URL
3	Selenium, Playwright (Bước 5–6)	2 script chạy được trên 20 URL
4	Đo hiệu năng, kiểm tra độ đúng, đếm dòng code (Bước 7–9)	Bảng kết quả thô RQ1–RQ3

Sản phẩm cuối của nhóm: repo GitHub gồm 4 script, file `benchmark.db` (SQLite), 3 bảng kết quả, và bản thảo bài báo 6–8 trang.

2. Lý thuyết cần nắm

Bốn công cụ chia thành hai nhóm: nhóm **tải HTML** (Requests + BeautifulSoup, Scrapy) chỉ nhận mã nguồn máy chủ gửi về, và nhóm **điều khiển trình duyệt** (Selenium, Playwright) mở một trình duyệt thật, chạy JavaScript rồi mới đọc trang. Nhóm đầu nhanh và nhẹ, nhóm sau chậm và nặng nhưng thấy được nội dung do JavaScript tạo ra. Toàn bộ đề tài là đo xem "nhanh hơn bao nhiêu" và "thấy được nhiều hơn bao nhiêu".

flowchart LR

```

A[Script Python] --> B{Nhóm công cụ}
B -->|Requests / Scrapy| C[Gửi HTTP request]
C --> D[Nhận HTML thô<br/>chưa chạy JavaScript]
B -->|Selenium / Playwright| E[Mở Chromium chế độ ảnh]
E --> F[Chạy JavaScript<br/>bấm nút, cuộn trang]
D --> G[Trích xuất dữ liệu]
F --> G
G --> H[(SQLite)]

```

Nhánh trên không mở trình duyệt nên tiết kiệm tài nguyên; nhánh dưới phải tải cả ảnh, CSS, JavaScript của trang.

2.1. Bốn công cụ

Công cụ	Nhóm	Cách hoạt động	Điểm mạnh	Điểm yếu
Requests + BeautifulSoup	Tải HTML	<code>requests</code> tải trang, <code>BeautifulSoup</code> tìm thẻ HTML	Ít code, dễ học, tốn ít RAM	Không chạy JavaScript, tự xử lý lỗi và thử lại
Scrapy	Tải HTML	Framework: tự quản lý hàng đợi, thử lại, giới hạn tốc độ	Nhanh khi chạy song song, có sẵn tôn trọng <code>robots.txt</code>	Khái niệm mới (spider, callback), khó gỡ lỗi với người mới
Selenium	Trình duyệt	Điều khiển Chrome qua WebDriver	Tài liệu tiếng Việt nhiều, đã học ở chương 02	Chậm, tốn RAM, phải tự viết lệnh chờ

Công cụ	Nhóm	Cách hoạt động	Điểm mạnh	Điểm yếu
Playwright	Trình duyệt	Điều khiển Chromium qua giao thức riêng	Tự động chờ phần tử xuất hiện, API gọn	Vẫn tồn tại nguyên như mọi trình duyệt

2.2. Ba loại trang cần phân biệt (dùng cho RQ2)

- **Trang tĩnh (render từ máy chủ):** dữ liệu nằm sẵn trong mã nguồn HTML. Cách kiểm tra: bấm `Ctrl+U` (View Page Source), `Ctrl+F` tìm giá sản phẩm; tìm thấy là tĩnh. Trên Long Châu, trang chi tiết sản phẩm thuộc loại này.
- **Nội dung render bằng JavaScript:** thấy trên màn hình nhưng không có trong `Ctrl+U`. Nhóm tải HTML sẽ lấy thiếu phần này.
- **Trang tải thêm (nút "Xem thêm" / cuộn vô hạn):** chỉ hiển thị một phần, phần còn lại được tải khi bấm nút hoặc cuộn. Trên Long Châu, trang danh mục chỉ hiện khoảng 10–12 sản phẩm đầu, kèm nút "Xem thêm N sản phẩm". Nút này hoạt động giống cuộn vô hạn nên dùng thay cho loại trang đó.

2.3. Các đại lượng đo

Đại lượng	Định nghĩa	Cách đo	Trả lời RQ
Thời gian (giây)	Thời gian xử lý 1.000 trang, không tính thời gian nghỉ lịch sự giữa các request	<code>time.perf_counter()</code> quanh phần tải + trích xuất	RQ1
RAM đỉnh (MB)	Bộ nhớ lớn nhất trong lúc chạy, gồm cả tiến trình trình duyệt con	Thư viện <code>psutil</code> , lấy mẫu 0,5 giây/lần	RQ1
Tỷ lệ thành công (%)	Số trang tải được và có dữ liệu / tổng số trang	Đếm bằng SQL	RQ1, RQ2
Độ đúng (%)	Số trường lấy đúng / tổng số trường, so với bộ đáp án kiểm tra thủ công	SQL so khớp với bảng <code>dap_an</code>	RQ2
Độ đầy đủ (%)	Số sản phẩm lấy được / số sản phẩm trang công bố	So với con số trên nút "Xem thêm"	RQ2

Đại lượng	Định nghĩa	Cách đo	Trả lời RQ
Số dòng code	Dòng không trống, không phải chú thích	Script đếm ở Bước 9	RQ3

Vì sao không tính thời gian nghỉ: mỗi request phải cách nhau ít nhất 1 giây để không gây tải cho website. Nếu tính cả thời gian nghỉ, công cụ nào cũng mất trên 1.000 giây và kết quả so sánh mất ý nghĩa.

Vì sao chạy tuần tự (1 request mỗi lần): Scrapy mặc định chạy song song nên sẽ "thăng" một cách không công bằng. Thí nghiệm chính cho cả 4 công cụ chạy tuần tự; chạy song song là phần mở rộng tùy chọn.

3. Chuẩn bị: môi trường, khảo sát website, quy tắc thu thập

Trước khi viết dòng code nào, nhóm cài đủ 4 công cụ, tạo repo Git, và tự tay khảo sát cấu trúc website Long Châu trong khoảng 30 phút.

3.1. Cài đặt

Dùng Python 3.10 trở lên. Mở Terminal tại thư mục dự án và chạy:

```
python -m venv .venv
# Windows: .venv\Scripts\activate | macOS/Linux: source
.venv/bin/activate
pip install requests beautifulsoup4 lxml scrapy selenium playwright psutil
playwright install chromium
pip freeze > requirements.txt # ghi lại phiên bản, bắt buộc nêu trong
bài báo
```

Selenium từ phiên bản 4.6 tự tải ChromeDriver (Selenium Manager), máy chỉ cần cài sẵn Google Chrome.

3.2. Cấu trúc thư mục và Git

```
osds-benchmark/
├── cau_hinh.py           # cau hinh chung: duong dan DB, thoi gian nghi,
User-Agent
├── tien_ich.py           # ham dung chung: tao bang, trích xuất trường,
ghi SQLite
├── xem_them.py           # bam nut "Xem them" (dung o Buoc 2 va Buoc 8)
├── b2_lay_danh_sach_url.py
├── cong_cu_requests.py
├── cong_cu_scrapy.py
├── cong_cu_selenium.py
├── cong_cu_playwright.py
├── do_hieu_nang.py       # chay tung cong cu va do thoi gian, RAM
├── b8_trang_xem_them.py
├── b8_trang_js.py
├── dem_dong_code.py
├── khao_sat.md          # ket qua khao sat website + cau hinh may
├── benchmark.db         # KHONG dua len Git neu > 50 MB
└── requirements.txt
```

```
git init
git add . && git commit -m "Khoi tao du an benchmark"
```

Mỗi bước thực hành bên dưới kết thúc bằng một lần `git commit`. Lịch sử commit là bằng chứng quá trình làm việc của nhóm.

3.3. Khảo sát website Long Châu (làm thủ công)

Mục tiêu của bước này là biết dữ liệu nằm ở đâu trước khi viết code. Ghi kết quả vào file `khao_sat.md` trong repo.

1. Mở một trang danh mục, ví dụ [Vitamin & Khoáng chất](#). Ghi lại: trang hiện bao nhiêu sản phẩm, nút "Xem thêm N sản phẩm" ghi con số bao nhiêu. Tổng số sản phẩm của danh mục = số đang hiện + N.
2. Bấm `Ctrl+U` trên trang danh mục, `Ctrl+F` tìm tên sản phẩm thứ 1 và tên sản phẩm xuất hiện sau khi bấm "Xem thêm". Sản phẩm nào không có trong mã nguồn là do JavaScript tải thêm.
3. Mở một trang chi tiết sản phẩm, ví dụ [Active Nerves New Nordic](#). Lần lượt tìm trong `Ctrl+U`: tên sản phẩm, giá (dạng 468.000đ / Hộp), thương hiệu, số đăng ký, số lượt đánh giá và bình luận. Trường nào không có trong mã nguồn thì đánh dấu "JS".

4. Bấm **F12** → tab Elements, chuột phải vào tên và giá sản phẩm → Inspect, ghi lại thẻ HTML chứa chúng. Tên sản phẩm nằm trong thẻ `<h1>`; thương hiệu là liên kết có đường dẫn chứa `/thuong-hieu/`.
5. Trong **Ctrl+U**, tìm chuỗi `__NEXT_DATA__`. Nếu có, trang dùng Next.js và thẻ này chứa dữ liệu dạng JSON; ghi nhận lại, nhưng bài thực hành vẫn trích xuất từ HTML để 4 công cụ dùng cùng một cách làm.

Kết quả khảo sát cần có: bảng liệt kê từng trường dữ liệu, nằm trong HTML hay do JS tạo. Bảng này chính là cơ sở phân loại trang cho RQ2 và đưa thẳng vào phần Phương pháp của bài báo. Cấu trúc website có thể thay đổi theo thời gian, nên nhóm phải ghi ngày khảo sát.

3.4. Quy tắc thu thập (bắt buộc)

- **Kiểm tra robots.txt trước** bằng đoạn code bên dưới; đường dẫn nào bị cấm thì không thu thập. Đọc thêm [Quy chế hoạt động](#) của website.
- **Nghỉ tối thiểu 1 giây giữa hai request**, chạy ngoài giờ cao điểm (sau 22h hoặc sáng sớm).
- **Chỉ lấy thông tin sản phẩm công khai** (tên, giá, đơn vị, thương hiệu, số đăng ký). **Không** thu thập bình luận, tên người đánh giá hay bất kỳ thông tin cá nhân nào.
- **Không đăng nhập, không thêm vào giỏ hàng, không gửi form.**
- **Tổng số request cả nhóm không quá khoảng 6.000** (4 công cụ × 1.000 trang + phần lặp lại nhỏ ở Bước 8). Nếu nhận mã lỗi 403 hoặc 429, dừng lại và báo giảng viên.

```
# kiem_tra_robots.py
from urllib import robotparser

rp = robotparser.RobotFileParser()
rp.set_url("https://nhathuoclongchau.com.vn/robots.txt")
rp.read()

url_thu = "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/vitamin-
khoang-chat"
print("Duoc phep thu thap:", rp.can_fetch("*", url_thu))
print("Crawl-delay:", rp.crawl_delay("*"))      # neu co, dung gia tri nay
thay cho 1 giay
print("Sitemap:", rp.site_maps())                # neu co sitemap, Bước 1 se
don gian hon
```

4. Thực hành từng bước

Nguyên tắc thiết kế quan trọng nhất: **cả 4 công cụ dùng chung một hàm trích xuất `trich_xuat(html)`** và chung một danh sách URL. Nhờ vậy mọi khác biệt về thời gian, RAM, độ đúng chỉ đến từ cách mỗi công cụ **tải trang**, và số dòng code đếm ở RQ3 cũng chỉ là phần code riêng của công cụ. Đây là điểm cần nêu rõ trong phần Phương pháp của bài báo.

Mỗi bước gồm: viết code → chạy thử trên 20 URL → kiểm tra bằng SQL → `git commit`.

Bước 1. Code dùng chung

File `cau_hinh.py` — mọi thông số nằm một chỗ để 4 công cụ chạy cùng điều kiện:

```
# cau_hinh.py - cau hinh dung chung cho ca 4 cong cu
DB = "benchmark.db"          # file SQLite lưu toàn bộ kết quả
NGHI = 1.0                   # số giây nghỉ giữa 2 request (lịch sử với
                             # website)
TIMEOUT = 20                 # số giây tối đa cho 1 trang
SO_URL = 1000                # số trang cần thu thập mỗi lần chạy
UA = ("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
      "(KHTML, like Gecko) Chrome/124.0 Safari/537.36 OSDS-student-project")
```

File `tien_ich.py` — tạo bảng SQLite, đọc URL, trích xuất 5 trường, ghi kết quả:

```
# tien_ich.py - ham dung chung: tao bang, doc URL, trích xuất, ghi kết quả
import re
import sqlite3
import sys
from datetime import datetime

from bs4 import BeautifulSoup

from cau_hinh import DB, SO_URL

MAU_GIA = re.compile(r"(\d{1,3}(\?:\.\d{3})+|\d+)\s*\d\s*/\s*([^\s\d]+)")
MAU_SO_DK = re.compile(r"Số đăng ký\s+(\S+)")

def ket_noi():
    con = sqlite3.connect(DB)
    con.executescript("""
        CREATE TABLE IF NOT EXISTS urls (
```

```

        id INTEGER PRIMARY KEY AUTOINCREMENT,
        url TEXT UNIQUE, danh_muc TEXT);
CREATE TABLE IF NOT EXISTS ket_qua (
    cong_cu TEXT, lan INTEGER, url TEXT, trang_thai TEXT,
    ten TEXT, gia INTEGER, don_vi TEXT, thuong_hieu TEXT, so_dang_ky
TEXT,
    thoi_gian_xu_ly REAL, thoi_diem TEXT,
    PRIMARY KEY (cong_cu, lan, url));
CREATE TABLE IF NOT EXISTS lan_chay (
    cong_cu TEXT, lan INTEGER, so_url INTEGER, so_trang_ok INTEGER,
    tong_thoi_gian REAL, thoi_gian_xu_ly REAL, ram_dinh_mb REAL,
thoi_diem TEXT);
    """)
    return con

def tham_so():
    """Doc tham so dong lenh: python cong_cu_x.py [so_url] [lan]"""
    so_url = int(sys.argv[1]) if len(sys.argv) > 1 else SO_URL
    lan = int(sys.argv[2]) if len(sys.argv) > 2 else 1
    return so_url, lan

def doc_url(con, so_url):
    rows = con.execute("SELECT url FROM urls ORDER BY id LIMIT ?", (so_url,))
    return [r[0] for r in rows]

def trich_xuat(html):
    """Ca 4 cong cu dung CHUNG ham nay, nen khac biet chi nam o cach tai
trang."""
    soup = BeautifulSoup(html, "lxml")
    h1 = soup.find("h1")
    ten = h1.get_text(" ", strip=True) if h1 else None

    van_ban = soup.get_text(" ", strip=True)
    if ten and ten in van_ban:                                # chi tim gia SAU ten san
pham
        van_ban_sau_ten = van_ban[van_ban.index(ten):]
    else:
        van_ban_sau_ten = van_ban

    m = MAU_GIA.search(van_ban_sau_ten)

```

```

gia = int(m.group(1).replace(".", "")) if m else None
don_vi = m.group(2) if m else None

th = soup.select_one('a[href*="/thuong-hieu/"]')
thuong_hieu = th.get_text(strip=True) if th else None

m2 = MAU_SO_DK.search(van_ban)
so_dang_ky = m2.group(1) if m2 else None

return {"ten": ten, "gia": gia, "don_vi": don_vi,
        "thuong_hieu": thuong_hieu, "so_dang_ky": so_dang_ky}

def luu_ket_qua(con, cong_cu, lan, url, du_lieu, thoi_gian, loi=None):
    if loi:
        trang_thai = "loi: " + loi
    elif du_lieu.get("ten") and du_lieu.get("gia"):
        trang_thai = "ok"
    else:
        trang_thai = "thieu_du_lieu"
    con.execute(
        "INSERT OR REPLACE INTO ket_qua VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
        (cong_cu, lan, url, trang_thai,
         du_lieu.get("ten"), du_lieu.get("gia"), du_lieu.get("don_vi"),
         du_lieu.get("thuong_hieu"), du_lieu.get("so_dang_ky"),
         thoi_gian, datetime.now().isoformat(timespec="seconds")))
    con.commit()

```

Hàm `trich_xuat` không dựa vào tên class CSS (hay thay đổi) mà dựa vào các mốc ổn định: thẻ `<h1>`, mẫu giá `468.000đ / Hộp`, liên kết `/thuong-hieu/` và cụm từ "Số đăng ký". Giá chỉ được tìm **sau** tên sản phẩm để không lấy nhầm giá của mục "Sản phẩm liên quan". Nếu khảo sát ở mục 3.3 cho thấy website đã đổi cách hiển thị, nhóm chỉ cần sửa hàm này.

File `xem_them.py` — bấm nút "Xem thêm N sản phẩm" cho đến hết, dùng cho Bước 2 và Bước 8:

```

# xem_them.py - ham dung chung de bam nut "Xem them N san pham" cho den het
import re
import time
from urllib.parse import urljoin, urlparse

# Phan tu SAU NHAT co chua ca "Xem them" va "san pham" (tranh bam nham nut

```

```

"Xem thêm" o bo loc)
XPATH_NUT = ("//*[contains(normalize-space(.), 'Xem thêm') and
contains(normalize-space(.), 'sản phẩm')
    " and not(//*[contains(normalize-space(.), 'Xem thêm') and
contains(normalize-space(.), 'sản phẩm'])])")
GOC_SAN_PHAM = ("/thuc-pham-chuc-nang/", "/duoc-my-pham/", "/cham-soc-ca-
nhan/", "/trang-thiet-bi-y-te/")
MAU_CON_LAI = re.compile(r"Xem thêm\s*(\d+)\s*sản phẩm")
SO_LAN_BAM_TOI_DA = 60

def loc_link_san_pham(ds_href, url_goc):
    """Giu lai link trang chi tiet san pham (.html) thuoc cac danh muc hang
    hoa."""
    ket_qua = set()
    for href in ds_href:
        if not href:
            continue
        day_du = urljoin(url_goc, href).split("?")[0]
        duong_dan = urlparse(day_du).path
        if duong_dan.endswith(".html") and
duong_dan.startswith(GOC_SAN_PHAM):
            ket_qua.add(day_du)
    return ket_qua

def so_con_lai(van_ban):
    """Doc so N trong 'Xem thêm N sản phẩm'; khong co nut thi tra ve 0."""
    m = MAU_CON_LAI.search(" ".join(van_ban.split()))
    return int(m.group(1)) if m else 0

def bam_het_playwright(trang, nghi=2.0):
    so_lan = 0
    while so_lan < SO_LAN_BAM_TOI_DA:
        nut = trang.locator(f"xpath={XPATH_NUT}")
        if nut.count() == 0:
            break
        nut.first.click()
        so_lan += 1
        time.sleep(nghi)
    # cho san pham moi tai xong +
    lich su voi website
    return so_lan

```

```
def bam_het_selenium(driver, nghi=2.0):
    from selenium.webdriver.common.by import By
    so_lan = 0
    while so_lan < SO_LAN_BAM_TOI_DA:
        nut = driver.find_elements(By.XPATH, XPATH_NUT)
        if not nut:
            break
        driver.execute_script("arguments[0].scrollIntoView({block:
'center'}); arguments[0].click();", nut[0])
        so_lan += 1
        time.sleep(nghi)
    return so_lan
```

Danh mục thuốc (/thuoc/) bị loại khỏi GOC_SAN_PHAM vì nhiều sản phẩm không hiển thị giá (cần tư vấn dược sĩ), sẽ làm sai lệch độ đúng.

Bước 2. Lấy danh sách 1.000 URL sản phẩm

Script mở lần lượt các trang danh mục, bấm "Xem thêm" đến hết, lấy link sản phẩm và dừng khi đủ 1.000 URL. Chạy một lần duy nhất; cả 4 công cụ sau đó dùng chung bảng urls.

```
# b2_lay_danh_sach_url.py - lay >= 1.000 URL san pham tu cac trang danh muc
(dung Playwright)
import time

from playwright.sync_api import sync_playwright

from cau_hinh import SO_URL, UA
from tien_ich import ket_noi
from xem_them import bam_het_playwright, loc_link_san_pham

DANH_MUC = [
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/vitamin-khoang-
chat",
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/ho-tro-tieu-hoa",
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/than-kinh-nao",
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/ho-tro-mien-dich-
tang-suc-de-khang",
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/co-xuong-khop",
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/bao-ve-mat",
```

```

"https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/lam-dep",
"https://nhathuoclongchau.com.vn/duoc-my-pham/cham-soc-da-mat",
"https://nhathuoclongchau.com.vn/duoc-my-pham/cham-soc-co-the",
"https://nhathuoclongchau.com.vn/duoc-my-pham/cham-soc-toc-da-dau",
"https://nhathuoclongchau.com.vn/cham-soc-ca-nhan/cham-soc-rang-mieng",
"https://nhathuoclongchau.com.vn/cham-soc-ca-nhan/ve-sinh-hang-ngay",
"https://nhathuoclongchau.com.vn/cham-soc-ca-nhan/cham-soc-me-va-be",
"https://nhathuoclongchau.com.vn/trang-thiet-bi-y-te/dung-cu-y-te",
"https://nhathuoclongchau.com.vn/trang-thiet-bi-y-te/theo-doi-suc-khoe",
"https://nhathuoclongchau.com.vn/trang-thiet-bi-y-te/so-cuu-giam-dau-tai-
cho",
]

con = ket_noi()
with sync_playwright() as p:
    trinh_duyet = p.chromium.launch(headless=True)
    trang = trinh_duyet.new_page(user_agent=UA)
    for dm in DANH_MUC:
        trang.goto(dm, wait_until="load")
        time.sleep(3)
        so_lan = bam_het_playwright(trang)
        ds_href = trang.eval_on_selector_all("a[href]", "els => els.map(e =>
e.getAttribute('href'))")
        links = loc_link_san_pham(ds_href, dm)
        for url in links:
            con.execute("INSERT OR IGNORE INTO urls(url, danh_muc) VALUES (?,
?), (url, dm))
            con.commit()
            tong = con.execute("SELECT COUNT(*) FROM urls").fetchone()[0]
            print(f"{dm}: bam {so_lan} lan, {len(links)} link -> tong {tong}
URL")
            if tong >= SO_URL:
                break
    trinh_duyet.close()
con.close()

```

Kiểm tra: `SELECT danh_muc, COUNT(*) FROM urls GROUP BY danh_muc`; phải ra tổng ≥ 1.000. Nếu chưa đủ, thêm danh mục con khác (lấy từ menu trang chủ) vào `DANH_MUC`. Nếu mỗi danh mục chỉ ra khoảng 10-12 link và "bấm 0 lần", nút "Xem thêm" đã đổi chữ; mở F12 xem lại và sửa `XPATH_NUT`.

```
git add . && git commit -m "Buoc 2: lay danh sach 1000 URL"
```

Bước 3. Công cụ 1 — Requests + BeautifulSoup

```
# cong_cu_requests.py - Requests + BeautifulSoup
import time

import requests

from cau_hinh import NGHI, TIMEOUT, UA
from tien_ich import doc_url, ket_noi, luu_ket_qua, tham_so, trich_xuat

so_url, lan = tham_so()
con = ket_noi()
phien = requests.Session()
phien.headers["User-Agent"] = UA

for url in doc_url(con, so_url):
    t0 = time.perf_counter()
    try:
        r = phien.get(url, timeout=TIMEOUT)
        r.raise_for_status()
        r.encoding = "utf-8" # website tiếng Việt: ép giải mã
        UTF-8
        du_lieu = trich_xuat(r.text)
        luu_ket_qua(con, "requests", lan, url, du_lieu, time.perf_counter() -
t0)
    except Exception as e:
        luu_ket_qua(con, "requests", lan, url, {}, time.perf_counter() - t0,
type(e).__name__)
        time.sleep(NGHI)

con.close()
```

Chạy thử: `python cong_cu_requests.py 20 0` (20 URL, lần chạy số 0 = chạy thử). **Kiểm tra:** `SELECT trang_thai, COUNT(*) FROM ket_qua WHERE cong_cu='requests' AND lan=0 GROUP BY trang_thai;` — phần lớn phải là `ok`. Mở 3 sản phẩm bất kỳ trên trình duyệt và so với dữ liệu trong bảng.

Dòng `time.sleep(NGHI)` nằm ngoài đoạn đo thời gian, nên `thoi_gian_xu_ly` chỉ gồm tải + trích xuất + ghi DB.

Bước 4. Công cụ 2 — Scrapy

Scrapy được cấu hình chạy tuần tự (`CONCURRENT_REQUESTS = 1`) và tự nghỉ `DOWNLOAD_DELAY` giây. Thời gian mỗi trang = `download_latency` (Scrapy tự đo thời gian tải) + thời gian trích xuất.

```
# cong_cu_scrapy.py - Scrapy (chạy tuần tự để so sánh công bằng)
import time

import scrapy
from scrapy.crawler import CrawlerProcess

from cau_hinh import NGHI, TIMEOUT, UA
from tien_ich import doc_url, ket_noi, luu_ket_qua, tham_so, trich_xuat

so_url, lan = tham_so()

class LongChauSpider(scrapy.Spider):
    name = "longchau"
    custom_settings = {
        "CONCURRENT_REQUESTS": 1,          # tuần tự, giống 3 công cụ còn lại
        "DOWNLOAD_DELAY": NGHI,
        "ROBOTSTXT_OBEY": True,
        "USER_AGENT": UA,
        "DOWNLOAD_TIMEOUT": TIMEOUT,
        "LOG_LEVEL": "WARNING",
    }

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.con = ket_noi()

    async def start(self):                    # Scrapy >= 2.13
        for yeu_cau in self.start_requests():
            yield yeu_cau

    def start_requests(self):                # Scrapy < 2.13
        for url in doc_url(self.con, so_url):
            yield scrapy.Request(url, callback=self.parse,
                                errback=self.bao_loi)

    def parse(self, response):
```

```

        t0 = time.perf_counter()
        du_lieu = trich_xuat(response.text)
        thoi_gian = response.meta.get("download_latency", 0) +
(time.perf_counter() - t0)
        luu_ket_qua(self.con, "scrapy", lan, response.url, du_lieu,
thoi_gian)

    def bao_loi(self, failure):
        luu_ket_qua(self.con, "scrapy", lan, failure.request.url, {}, None,
failure.type.__name__)

    def closed(self, reason):
        self.con.close()

process = CrawlerProcess()
process.crawl(LongChauSpider)
process.start()

```

Chạy thử: `python cong_cu_scrapy.py 20 0`. Scrapy từ bản 2.13 dùng hàm `start()` thay cho `start_requests()`; code trên có cả hai để chạy được với mọi phiên bản. Scrapy đã bật `ROBOTSTXT_OBEY` nên tự bỏ qua URL bị robots.txt cấm.

```
git add . && git commit -m "Buoc 3-4: Requests va Scrapy"
```

Bước 5. Công cụ 3 — Selenium

```

# cong_cu_selenium.py - Selenium điều khiển Chrome an (headless)
import time

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

from cau_hinh import NGHI, TIMEOUT, UA
from tien_ich import doc_url, ket_noi, luu_ket_qua, tham_so, trich_xuat

so_url, lan = tham_so()
con = ket_noi()

tuy_chon = webdriver.ChromeOptions()

```

```

tuy_chon.add_argument("--headless=new")
tuy_chon.add_argument(f"--user-agent={UA}")
driver = webdriver.Chrome(options=tuy_chon)
driver.set_page_load_timeout(TIMEOUT)

for url in doc_url(con, so_url):
    t0 = time.perf_counter()
    try:
        driver.get(url)
        WebDriverWait(driver,
10).until(EC.presence_of_element_located((By.TAG_NAME, "h1")))
        du_lieu = trich_xuat(driver.page_source)
        luu_ket_qua(con, "selenium", lan, url, du_lieu, time.perf_counter() -
t0)
    except Exception as e:
        luu_ket_qua(con, "selenium", lan, url, {}, time.perf_counter() - t0,
type(e).__name__)
        time.sleep(NGHI)

driver.quit()
con.close()

```

Chạy thử: `python cong_cu_selenium.py 20 0`. Lưu ý cho bài báo: Selenium không đọc được mã trạng thái HTTP, nên trang lỗi 404 sẽ bị ghi là `thieu_du_lieu` thay vì `loi`. Đây là một khác biệt về khả năng xử lý lỗi đáng nêu trong phần Thảo luận.

Bước 6. Công cụ 4 — Playwright

```

# cong_cu_playwright.py - Playwright điều khiển Chromium an (headless)
import time

from playwright.sync_api import sync_playwright

from cau_hinh import NGHI, TIMEOUT, UA
from tien_ich import doc_url, ket_noi, luu_ket_qua, tham_so, trich_xuat

so_url, lan = tham_so()
con = ket_noi()

with sync_playwright() as p:
    trinh_duyet = p.chromium.launch(headless=True)
    trang = trinh_duyet.new_page(user_agent=UA)

```

```

    for url in doc_url(con, so_url):
        t0 = time.perf_counter()
        try:
            phan_hoi = trang.goto(url, timeout=TIMEOUT * 1000,
wait_until="load")
            if phan_hoi and phan_hoi.status >= 400:
                raise RuntimeError(f"HTTP{phan_hoi.status}")
            trang.wait_for_selector("h1", timeout=10_000)
            du_lieu = trich_xuat(trang.content())
            luu_ket_qua(con, "playwright", lan, url, du_lieu,
time.perf_counter() - t0)
        except Exception as e:
            luu_ket_qua(con, "playwright", lan, url, {}, time.perf_counter()
- t0, type(e).__name__)
            time.sleep(NGHI)
            trinh_duyet.close()

con.close()

```

Chạy thử: `python cong_cu_playwright.py 20 0`. Khác Selenium, Playwright đọc được mã trạng thái HTTP qua `phan_hoi.status`.

```
git add . && git commit -m "Buoc 5-6: Selenium va Playwright"
```

Bước 7. Đo thời gian và RAM (RQ1)

Script này chạy từng công cụ thành một tiến trình riêng, cứ 0,5 giây đo RAM một lần (cộng cả tiến trình trình duyệt con), rồi ghi vào bảng `lan_chay`. Nếu chỉ đo RAM của tiến trình Python, Selenium và Playwright sẽ trông "nhẹ" hơn thực tế rất nhiều, vì phần lớn bộ nhớ nằm ở tiến trình Chrome.

```

# do_hieu_nang.py - chay tung cong cu nhu mot tien trinh rieng, do thoi gian
va RAM dinh
# Cach dung: python do_hieu_nang.py <so_url> <lan> [cong_cu ...]
import sys
import time
from datetime import datetime

import psutil

from tien_ich import ket_noi

```

```

SCRIPT = {
    "requests": "cong_cu_requests.py",
    "scrapy": "cong_cu_scrapy.py",
    "selenium": "cong_cu_selenium.py",
    "playwright": "cong_cu_playwright.py",
}

def ram_mb(tien_trinh):
    """RAM của tiến trình + tất cả tiến trình con (trình duyệt, driver)."""
    tong = 0
    try:
        ds = [tien_trinh] + tien_trinh.children(recursive=True)
    except psutil.NoSuchProcess:
        return 0
    for tt in ds:
        try:
            tong += tt.memory_info().rss
        except psutil.NoSuchProcess:
            pass
    return tong / 1024 / 1024

def chay(cong_cu, so_url, lan):
    print(f"Dang chay {cong_cu} - {so_url} URL - lan {lan} ...")
    t0 = time.perf_counter()
    tien_trinh = psutil.Popen([sys.executable, SCRIPT[cong_cu], str(so_url),
str(lan)])
    ram_dinh = 0
    while tien_trinh.poll() is None:
        ram_dinh = max(ram_dinh, ram_mb(tien_trinh))
        time.sleep(0.5)
    tong_thoi_gian = time.perf_counter() - t0

    con = ket_noi()
    so_ok, tg_xu_ly = con.execute(
        "SELECT SUM(trang_thai = 'ok'), SUM(thoi_gian_xu_ly) FROM ket_qua "
        "WHERE cong_cu = ? AND lan = ?", (cong_cu, lan)).fetchone()
    con.execute("INSERT INTO lan_chay VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
        (cong_cu, lan, so_url, so_ok, round(tong_thoi_gian, 1),
        round(tg_xu_ly or 0, 1), round(ram_dinh, 1),
        datetime.now().isoformat(timespec="seconds")))
    con.commit()

```

```

con.close()
print(f"  xong: {so_ok or 0} trang ok, xu ly {tg_xu_ly or 0:.1f} s, RAM
dinh {ram_dinh:.0f} MB")

if __name__ == "__main__":
    so_url = int(sys.argv[1]) if len(sys.argv) > 1 else 1000
    lan = int(sys.argv[2]) if len(sys.argv) > 2 else 1
    ds_cong_cu = sys.argv[3:] or list(SCRIPT)
    for cc in ds_cong_cu:
        chay(cc, so_url, lan)

```

Kịch bản chạy thí nghiệm:

```

# Lan 1: 1.000 trang, moi cong cu chay mot dem (Selenium/Playwright co the
mat 1-2 gio)
python do_hieu_nang.py 1000 1 requests
python do_hieu_nang.py 1000 1 scrapy
python do_hieu_nang.py 1000 1 selenium
python do_hieu_nang.py 1000 1 playwright

# Lan 2-4: lap lai tren 100 trang dau, DAO THU TU cong cu de giam anh huong
cua mang
python do_hieu_nang.py 100 2 requests scrapy selenium playwright
python do_hieu_nang.py 100 3 playwright selenium scrapy requests
python do_hieu_nang.py 100 4 selenium requests playwright scrapy

```

Trong lúc chạy, máy chỉ mở đúng chương trình đo, cắm sạc (laptop), dùng cùng một mạng. Ghi cấu hình máy (CPU, RAM, hệ điều hành) và loại mạng vào `khao_sat.md`; đây là thông tin bắt buộc trong phần Phương pháp.

Bước 8. Kiểm tra độ đúng theo loại trang (RQ2)

RQ2 có ba phép đo, ứng với ba loại trang ở mục 2.2.

8a. Trang tĩnh — độ đúng 5 trường trên trang chi tiết sản phẩm. Nhóm chọn ngẫu nhiên 50 URL, tự mở từng trang và gõ đáp án bằng [DB Browser for SQLite](#). Nên nhập đáp án trong cùng ngày chạy Lần 1 vì giá có thể thay đổi.

```

CREATE TABLE dap_an (url TEXT PRIMARY KEY, ten TEXT, gia INTEGER, don_vi
TEXT,
                        thuong_hieu TEXT, so_dang_ky TEXT);
INSERT INTO dap_an(url) SELECT url FROM urls ORDER BY RANDOM() LIMIT 50;
-- Mo DB Browser, dien tay 5 cot cho 50 dong (trang khong co so dang ky thi
de trong)

-- Do dung theo tung cong cu (dung IS de hai gia tri cung rong van tinh la
dung)
SELECT k.cong_cu,
       ROUND(100.0 * (SUM(k.ten IS d.ten) + SUM(k.gia IS d.gia) +
SUM(k.don_vi IS d.don_vi)
           + SUM(k.thuong_hieu IS d.thuong_hieu) + SUM(k.so_dang_ky IS
d.so_dang_ky))
           / (5 * COUNT(*)), 1) AS do_dung_phan_tram
FROM ket_qua k JOIN dap_an d ON k.url = d.url
WHERE k.lan = 1
GROUP BY k.cong_cu;

```

8b. Trang tải thêm — nút "Xem thêm" của Long Châu. Script đếm số sản phẩm mỗi công cụ lấy được trên 10 trang danh mục, so với tổng số trang công bố (số đang hiện + N trên nút).

```

# b8_trang_xem_them.py - RQ2: moi cong cu lay duoc bao nhieu % san pham tren
trang co nut "Xem them"
# Cach dung: python b8_trang_xem_them.py requests | selenium | playwright
import sys
import time
from datetime import datetime

import requests
from bs4 import BeautifulSoup

from cau_hinh import NGHI, TIMEOUT, UA
from tien_ich import ket_noi
from xem_them import bam_het_playwright, bam_het_selenium, loc_link_san_pham,
so_con_lai

DANH_MUC_TEST = [
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/vitamin-khoang-
chat",
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/ho-tro-tieu-hoa",

```

```

    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/than-kinh-nao",
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/ho-tro-mien-dich-
tang-suc-de-khang",
    "https://nhathuoclongchau.com.vn/thuc-pham-chuc-nang/co-xuong-khop",
    "https://nhathuoclongchau.com.vn/duoc-my-pham/cham-soc-da-mat",
    "https://nhathuoclongchau.com.vn/duoc-my-pham/cham-soc-co-the",
    "https://nhathuoclongchau.com.vn/cham-soc-ca-nhan/cham-soc-rang-mieng",
    "https://nhathuoclongchau.com.vn/trang-thiet-bi-y-te/dung-cu-y-te",
    "https://nhathuoclongchau.com.vn/trang-thiet-bi-y-te/theo-doi-suc-khoe",
]

```

```

def dem_requests(url):
    r = requests.get(url, headers={"User-Agent": UA}, timeout=TIMEOUT)
    r.encoding = "utf-8"
    soup = BeautifulSoup(r.text, "lxml")
    hrefs = [a.get("href") for a in soup.find_all("a")]
    ban_dau = len(loc_link_san_pham(hrefs, url))
    return ban_dau, so_con_lai(soup.get_text(" ")), ban_dau          # không
bam duoc nut

```

```

def dem_playwright(url, trang):
    trang.goto(url, wait_until="load")
    time.sleep(3)
    lay_href = "els => els.map(e => e.getAttribute('href'))"
    ban_dau = len(loc_link_san_pham(trang.eval_on_selector_all("a[href]",
lay_href), url))
    con_lai = so_con_lai(trang.inner_text("body"))
    bam_het_playwright(trang)
    sau_cung = len(loc_link_san_pham(trang.eval_on_selector_all("a[href]",
lay_href), url))
    return ban_dau, con_lai, sau_cung

```

```

def dem_selenium(url, driver):
    from selenium.webdriver.common.by import By
    driver.get(url)
    time.sleep(3)
    lay = lambda: [a.get_attribute("href") for a in
driver.find_elements(By.CSS_SELECTOR, "a[href]")]
    ban_dau = len(loc_link_san_pham(lay(), url))
    con_lai = so_con_lai(driver.find_element(By.TAG_NAME, "body").text)

```

```

    bam_het_selenium(driver)
    sau_cung = len(loc_link_san_pham(lay(), url))
    return ban_dau, con_lai, sau_cung

def luu(con, cong_cu, url, ket_qua):
    ban_dau, con_lai, lay_duoc = ket_qua
    con.execute("INSERT INTO xem_them VALUES (?, ?, ?, ?, ?, ?)",
                (cong_cu, url, ban_dau, con_lai, lay_duoc,
                 datetime.now().isoformat(timespec="seconds")))
    con.commit()
    print(f"{cong_cu}: {url} -> ban dau {ban_dau}, con lai {con_lai}, lay
    duoc {lay_duoc}")

cong_cu = sys.argv[1] if len(sys.argv) > 1 else "requests"
con = ket_noi()
con.execute("""CREATE TABLE IF NOT EXISTS xem_them (
    cong_cu TEXT, danh_muc TEXT, so_ban_dau INTEGER, so_con_lai_cong_bo
    INTEGER,
    so_lay_duoc INTEGER, thoi_diem TEXT)""")

if cong_cu == "requests":
    for url in DANH_MUC_TEST:
        luu(con, cong_cu, url, dem_requests(url))
        time.sleep(NGHI)
elif cong_cu == "playwright":
    from playwright.sync_api import sync_playwright
    with sync_playwright() as p:
        trinh_duyet = p.chromium.launch(headless=True)
        trang = trinh_duyet.new_page(user_agent=UA)
        for url in DANH_MUC_TEST:
            luu(con, cong_cu, url, dem_playwright(url, trang))
        trinh_duyet.close()
elif cong_cu == "selenium":
    from selenium import webdriver
    tuy_chon = webdriver.ChromeOptions()
    tuy_chon.add_argument("--headless=new")
    tuy_chon.add_argument(f"--user-agent={UA}")
    driver = webdriver.Chrome(options=tuy_chon)

```

```

for url in DANH_MUC_TEST:
    luu(con, cong_cu, url, dem_selenium(url, driver))
driver.quit()
con.close()

```

Với Scrapy, không cần viết thêm: Scrapy nhận cùng HTML thô như Requests. Nhóm kiểm chứng bằng lệnh `scrapy shell "<url danh mục>"`, sau đó gõ `len(set(response.css('a[href$=".html"]::attr(href)').getall()))` và so gần đúng với cột `so_ban_dau` của Requests.

```

SELECT cong_cu,
       SUM(so_lay_duoc) AS lay_duoc,
       SUM(so_ban_dau + so_con_lai_cong_bo) AS cong_bo,
       ROUND(100.0 * SUM(so_lay_duoc) / SUM(so_ban_dau + so_con_lai_cong_bo),
1) AS do_day_du_phan_tram
FROM xem_them GROUP BY cong_cu;

```

8c. Đối chứng: trang render hoàn toàn bằng JavaScript và trang cuộn vô hạn thật. Long Châu render phần lớn nội dung từ máy chủ và dùng nút bấm thay cho cuộn. Để RQ2 đủ ba loại trang, nhóm chạy thêm trên website luyện tập quotes.toscrape.com, được xây dựng riêng để tập thu thập dữ liệu: mục `/js/` render trích dẫn bằng JavaScript, mục `/scroll` tải thêm khi cuộn.

```

# b8_trang_js.py - RQ2 doi chung: trang render bang JavaScript va trang cuon
# vo han
# Dung website luyen tap quotes.toscrape.com (xay dung de tap thu thap du
# lieu)
# Cach dung: python b8_trang_js.py requests | selenium | playwright
import sys
import time

import requests
from bs4 import BeautifulSoup

from cau_hinh import NGHI, TIMEOUT, UA
from tien_ich import ket_noi

GOC = "https://quotes.toscrape.com"
TRANG_JS = [f"{GOC}/js/page/{i}/" for i in range(1, 11)] # 10 trang x 10
trich dan
TRANG_CUON = f"{GOC}/scroll" # 100 trich
dan, cuon de tai them

```

```

def cuon_het_playwright(trang):
    truoc = -1
    while True:
        so = trang.locator("div.quote").count()
        if so == truoc:
            return so
        truoc = so
        trang.mouse.wheel(0, 20000)
        time.sleep(NGHI + 1)

def cuon_het_selenium(driver):
    from selenium.webdriver.common.by import By
    truoc = -1
    while True:
        so = len(driver.find_elements(By.CSS_SELECTOR, "div.quote"))
        if so == truoc:
            return so
        truoc = so
        driver.execute_script("window.scrollTo(0,
document.body.scrollHeight);")
        time.sleep(NGHI + 1)

cong_cu = sys.argv[1] if len(sys.argv) > 1 else "requests"
ket_qua = [] # (loai_trang, url,
so_ky_vong, so_lay_duoc)

if cong_cu == "requests":
    for url in TRANG_JS + [TRANG_CUON]:
        html = requests.get(url, headers={"User-Agent": UA},
timeout=TIMEOUT).text
        so = len(BeautifulSoup(html, "lxml").select("div.quote"))
        loai = "cuon_vo_han" if url == TRANG_CUON else "render_js"
        ket_qua.append((loai, url, 100 if loai == "cuon_vo_han" else 10, so))
        time.sleep(NGHI)
elif cong_cu == "playwright":
    from playwright.sync_api import sync_playwright
    with sync_playwright() as p:
        trinh_duyet = p.chromium.launch(headless=True)
        trang = trinh_duyet.new_page(user_agent=UA)

```

```

        for url in TRANG_JS:
            trang.goto(url, wait_until="load")
            trang.wait_for_selector("div.quote", timeout=10_000)
            ket_qua.append(("render_js", url, 10,
trang.locator("div.quote").count()))
            time.sleep(NGHI)
            trang.goto(TRANG_CUON, wait_until="load")
            trang.wait_for_selector("div.quote", timeout=10_000)
            ket_qua.append(("cuon_vo_han", TRANG_CUON, 100,
cuon_het_playwright(trang)))
            trinh_duyet.close()
elif cong_cu == "selenium":
    from selenium import webdriver
    from selenium.webdriver.common.by import By
    from selenium.webdriver.support import expected_conditions as EC
    from selenium.webdriver.support.ui import WebDriverWait
    tuy_chon = webdriver.ChromeOptions()
    tuy_chon.add_argument("--headless=new")
    driver = webdriver.Chrome(options=tuy_chon)
    cho = WebDriverWait(driver, 10)
    for url in TRANG_JS:
        driver.get(url)
        cho.until(EC.presence_of_element_located((By.CSS_SELECTOR,
"div.quote"))))
        ket_qua.append(("render_js", url, 10,
len(driver.find_elements(By.CSS_SELECTOR, "div.quote"))))
        time.sleep(NGHI)
        driver.get(TRANG_CUON)
        cho.until(EC.presence_of_element_located((By.CSS_SELECTOR, "div.quote"))))
        ket_qua.append(("cuon_vo_han", TRANG_CUON, 100,
cuon_het_selenium(driver)))
        driver.quit()

con = ket_noi()
con.execute("""CREATE TABLE IF NOT EXISTS trang_js (
    cong_cu TEXT, loai_trang TEXT, url TEXT, so_ky_vong INTEGER, so_lay_duoc
INTEGER)""")
con.executemany("INSERT INTO trang_js VALUES (?, ?, ?, ?, ?)", [(cong_cu, *k) for
k in ket_qua])
con.commit()
con.close()
for k in ket_qua:
    print(cong_cu, k)

```

```
SELECT cong_cu, loai_trang,
       ROUND(100.0 * SUM(so_lay_duoc) / SUM(so_ky_vong), 1) AS
ty_le_lay_dung_phan_tram
FROM trang_js GROUP BY cong_cu, loai_trang;
```

Bước 9. Đếm dòng code và đánh giá độ dễ học (RQ3)

```
# dem_dong_code.py - dem dong code co nghĩa (bỏ dòng trong và dòng chú thích)
FILES = {
    "requests": "cong_cu_requests.py",
    "scrapy": "cong_cu_scrapy.py",
    "selenium": "cong_cu_selenium.py",
    "playwright": "cong_cu_playwright.py",
}

for cong_cu, ten_file in FILES.items():
    with open(ten_file, encoding="utf-8") as f:
        dong = [d.strip() for d in f]
        so_dong = sum(1 for d in dong if d and not d.startswith("#"))
        print(f"{cong_cu:<12}{so_dong:>5} dong")
```

Số dòng code mới chỉ là một nửa câu trả lời. Nửa còn lại "công cụ nào phù hợp người mới học" cần dữ liệu từ chính người học, thu theo hai cách đơn giản:

- **Nhật ký thời gian:** mỗi thành viên ghi số phút từ lúc bắt đầu đến lúc script chạy được lần đầu trên 20 URL, và số lỗi gặp phải, cho từng công cụ.
- **Phiếu đánh giá ngắn:** nhờ 15–20 bạn cùng lớp làm lại Bước 3–6 với code mẫu, rồi chấm từ 1 (rất khó) đến 5 (rất dễ) cho 3 câu: dễ cài đặt, dễ hiểu code, dễ sửa khi lỗi. Nhập vào bảng `danh_gia(nguai, cong_cu, cai_dat, hieu_code, sua_loi)` và tính bằng `AVG` theo công cụ.

```
git add . && git commit -m "Buoc 7-9: do hieu nang, do dung, dong code"
```

5. Tổng hợp kết quả và viết bài báo

Toàn bộ số liệu của bài báo lấy từ `benchmark.db` bằng các truy vấn SQL dưới đây, điền vào ba bảng mẫu. Các ô `...` là chỗ nhóm điền số liệu thật của mình; không điền số ước đoán.

5.1. RQ1 — Thời gian và RAM

```
-- Lan 1: 1.000 trang
SELECT cong_cu, so_url, so_trang_ok,
       ROUND(100.0 * so_trang_ok / so_url, 1) AS ty_le_thanh_cong_pct,
       thoi_gian_xu_ly AS xu_ly_giay,
       ROUND(thoi_gian_xu_ly / so_url, 3) AS giay_moi_trang,
       ram_dinh_mb
FROM lan_chay WHERE lan = 1 ORDER BY xu_ly_giay;

-- Lan 2-4: 100 trang x 3 lan, lay trung binh va khoang dao dong
SELECT cong_cu,
       ROUND(AVG(thoi_gian_xu_ly), 1) AS tb_xu_ly_giay,
       MIN(thoi_gian_xu_ly) AS min_giay, MAX(thoi_gian_xu_ly) AS max_giay,
       ROUND(AVG(ram_dinh_mb), 0) AS tb_ram_mb
FROM lan_chay WHERE lan BETWEEN 2 AND 4 GROUP BY cong_cu;
```

Bảng 1 (mẫu). Hiệu năng thu thập 1.000 trang chi tiết sản phẩm

Công cụ	Thời gian xử lý (giây)	Giây/trang	RAM đỉnh (MB)	Tỷ lệ thành công (%)	100 trang × 3 lần: TB (min-max) giây
Requests + BeautifulSoup	...	...	...	...	...
Scrapy	...	...	...	...	...
Selenium	...	...	...	...	...
Playwright	...	...	...	...	...

5.2. RQ2 — Độ đúng theo loại trang

Dùng các truy vấn ở Bước 8a, 8b, 8c. Thêm truy vấn độ đúng theo từng trường để biết công cụ sai ở đâu:

```

SELECT k.cong_cu,
       ROUND(100.0 * AVG(k.ten IS d.ten), 1) AS ten_pct,
       ROUND(100.0 * AVG(k.gia IS d.gia), 1) AS gia_pct,
       ROUND(100.0 * AVG(k.don_vi IS d.don_vi), 1) AS don_vi_pct,
       ROUND(100.0 * AVG(k.thuong_hieu IS d.thuong_hieu), 1) AS
thuong_hieu_pct,
       ROUND(100.0 * AVG(k.so_dang_ky IS d.so_dang_ky), 1) AS
so_dang_ky_pct
FROM ket_qua k JOIN dap_an d ON k.url = d.url
WHERE k.lan = 1 GROUP BY k.cong_cu;

```

Bảng 2 (mẫu). Tỷ lệ lấy đúng dữ liệu theo loại trang (%)

Công cụ	Trang tĩnh (độ đúng 5 trường, 50 trang)	Trang "Xem thêm" Long Châu (độ đầy đủ, 10 danh mục)	Trang render JS (quotes /js)	Trang cuộn vô hạn (quotes /scroll)
Requests + BeautifulSoup	...	...	...	...
Scrapy	...	... (kiểm bằng scrapy shell)	...	...
Selenium	...	...	...	...
Playwright	...	...	...	...

5.3. RQ3 — Số dòng code và độ dễ học

```

SELECT cong_cu, COUNT(*) AS so_nguoi,
       ROUND(AVG(cai_dat), 2) AS de_cai_dat, ROUND(AVG(hieu_code), 2) AS
de_hieu,
       ROUND(AVG(sua_loi), 2) AS de_sua_loi
FROM danh_gia GROUP BY cong_cu;

```

Bảng 3 (mẫu). Mức độ phù hợp với người mới học

Công cụ	Số dòng code	Phút đến lần chạy đầu (TB nhóm)	Dễ cài đặt (1-5)	Dễ hiểu code (1-5)	Dễ sửa lỗi (1-5)
Requests + BeautifulSoup	...	...	...	...	...

Công cụ	Số dòng code	Phút đến lần chạy đầu (TB nhóm)	Dễ cài đặt (1–5)	Dễ hiểu code (1–5)	Dễ sửa lỗi (1–5)
Scrapy	...	...	...	...	...
Selenium	...	...	...	...	...
Playwright	...	...	...	...	...

5.4. Dàn ý bài báo (6–8 trang)

Mục	Nội dung	Lấy từ đâu
Tóm tắt	Mục tiêu, website, 4 công cụ, 3 kết quả chính bằng con số	Viết sau cùng
1. Giới thiệu	Nhu cầu thu thập dữ liệu web ở Việt Nam; người mới khó chọn công cụ; 3 RQ	Mục 1 tài liệu này
2. Công trình liên quan	3–5 bài so sánh công cụ thu thập dữ liệu đã kiểm chứng trên Google Scholar	Nhóm tự tìm
3. Phương pháp	Website và ngày khảo sát; cấu hình máy, mạng, phiên bản thư viện; thiết kế dùng chung hàm trích xuất; cách đo; quy tắc thu thập	Mục 2.3, 3.3, 3.4, <code>requirements.txt</code> , <code>khao_sat.md</code>
4. Kết quả	Bảng 1, 2, 3 kèm 2–3 câu mô tả mỗi bảng	Mục 5.1–5.3
5. Thảo luận	Khi nào nên dùng công cụ nào; bảng khuyến nghị theo loại trang; hạn chế	Kết quả + phân hạn chế bên dưới
6. Kết luận	Trả lời ngắn gọn 3 RQ; hướng mở rộng	—

Hạn chế cần nêu trung thực trong bài: chỉ thử trên một website thương mại và một website luyện tập; chỉ một cấu hình máy; tốc độ mạng thay đổi theo giờ; chỉ so sánh ở chế độ tuần tự nên chưa thể hiện ưu thế chạy song song của Scrapy; đánh giá độ dễ học dựa trên mẫu nhỏ trong một lớp; website có thể đổi cấu trúc sau ngày khảo sát.

Công bố kèm bài: link repo GitHub gồm toàn bộ script, `requirements.txt`, `khao_sat.md` và các bảng kết quả dạng CSV. Không công bố lại toàn bộ nội dung trang của Long Châu; chỉ công bố số liệu đo đạc.

6. Bài tập, tiêu chí đánh giá và lỗi thường gặp

6.1. Bài tập theo buổi

- ☐ **Buổi 1:** hoàn thành `khao_sat.md` (bảng trường dữ liệu HTML/JS, ngày khảo sát, kết quả robots.txt) và bảng `urls` có ≥ 1.000 dòng.
- ☐ **Buổi 2:** `cong_cu_requests.py` và `cong_cu_scrapy.py` chạy được trên 20 URL, $\geq 90\%$ dòng có trạng thái `ok`.
- ☐ **Buổi 3:** `cong_cu_selenium.py` và `cong_cu_playwright.py` chạy được trên 20 URL; so sánh dữ liệu 20 URL giữa 4 công cụ bằng SQL và giải thích chỗ khác nhau (nếu có).
- ☐ **Buổi 4:** bảng `dap_an` 50 dòng; chạy thử `do_hieu_nang.py` 20 0; nộp ba bảng kết quả thô.
- ☐ **Tuần 5-10:** chạy thí nghiệm chính (Lần 1-4, Bước 8, phiếu đánh giá), viết bài báo theo dàn ý 5.4.

Bài tập mở rộng (không bắt buộc, dành cho nhóm khá):

1. Chạy Scrapy với `CONCURRENT_REQUESTS = 4` và `DOWNLOAD_DELAY = 1` trên 100 trang, so với chế độ tuần tự. Không tăng quá 4 luồng.
2. Chặn tải ảnh trong Playwright bằng `trang.route("**/*.{png,jpg,jpeg,webp,gif}", lambda r: r.abort())` và đo lại RAM, thời gian.
3. Nếu khảo sát tìm thấy thẻ `__NEXT_DATA__`, viết cách thứ 5: dùng Requests đọc JSON trong thẻ đó thay vì tìm trong HTML, rồi so độ đúng với 4 công cụ.

6.2. Tiêu chí đánh giá (thang 10)

Tiêu chí	Điểm	Đạt tối đa khi
Khảo sát website và tuân thủ quy tắc thu thập	1,0	<code>khao_sat.md</code> đầy đủ; không vượt giới hạn request; không thu thập thông tin cá nhân
4 script chạy được, dùng chung hàm trích xuất	2,0	Mỗi công cụ $\geq 90\%$ trang <code>ok</code> trên 1.000 URL
Thí nghiệm RQ1	1,5	Đủ Lần 1-4, có cấu hình máy và mạng

Tiêu chí	Điểm	Đạt tối đa khi
Thí nghiệm RQ2	1,5	Đủ 8a, 8b, 8c; bảng <code>dap_an</code> 50 dòng nhập thủ công
Thí nghiệm RQ3	1,0	Có số dòng code, nhật ký thời gian, ≥ 15 phiếu đánh giá
Bài báo	2,0	Đúng dàn ý, số liệu khớp với <code>benchmark.db</code> , tài liệu tham khảo có thật, nêu hạn chế
Git và tính tái lập	1,0	Commit sau mỗi bước; người khác clone repo và chạy lại được

6.3. Lỗi thường gặp

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
Tên sản phẩm bị lỗi font (ViÃãn)	Requests đoán sai bảng mã	Giữ dòng <code>r.encoding = "utf-8"</code> trước khi đọc <code>r.text</code>
Mọi dòng đều <code>thieu_du_lieu</code> , cột <code>gia</code> rỗng	Website đổi cách hiển thị giá	Mở <code>Ctrl+U</code> , xem lại chuỗi giá và sửa <code>MAU_GIA</code> trong <code>tien_ich.py</code>
Bước 2 báo "bấm 0 lần", mỗi danh mục chỉ 10–12 link	Chữ trên nút "Xem thêm" đã đổi	Inspect nút bằng F12 và sửa <code>XPATH_NUT</code> trong <code>xem_them.py</code>
Scrapy chạy xong ngay, bảng <code>ket_qua</code> trống	URL bị robots.txt cấm, hoặc bảng <code>urls</code> rỗng	Đổi <code>LOG_LEVEL</code> thành <code>"INFO"</code> để xem lý do; chạy lại Bước 2
Selenium báo lỗi không tìm thấy driver	Máy chưa cài Chrome hoặc Selenium quá cũ	Cài Google Chrome; <code>pip install -U selenium</code> (≥ 4.6)
Playwright báo <code>Executable doesn't exist</code>	Chưa tải trình duyệt	Chạy <code>playwright install chromium</code>

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
Mã lỗi 403 hoặc 429 liên tục	Bị website giới hạn do gửi quá nhanh	Dừng ngay , tăng <code>NGHI</code> lên 2–3 giây, báo giảng viên trước khi chạy lại
RAM của Selenium/Playwright quá thấp (vài chục MB)	Chỉ đo tiến trình Python, bỏ sót Chrome	Luôn chạy qua <code>do_hieu_nang.py</code> , không tự đo bằng Task Manager
<code>database is locked</code>	Mở <code>benchmark.db</code> trong DB Browser lúc script đang chạy	Đóng DB Browser (hoặc bấm Write Changes) rồi chạy lại